

1. 74164 시프트 레지스터를 활용하여 최근 8개의 입력이 1,1,1,1,0,0,0,0이면 출력 1을 내는 시스템을 설계하라.

[문제 1] 74164 시프트 레지스터를 활용하여 최근 8개의 입력이 1,1,1,0,0,0,0,0이면 출력 1을 내는 시스템을 설계하라.

1. 개념 분석

- 74164 특징: 8비트 직렬 입력-병렬 출력(SIPO) 시프트 레지스터입니다. 입력된 데이터가 클럭에 맞춰 Q_A 부터 Q_H 까지 차례대로 시프트됩니다.
- 입력 패턴: "최근 8개의 입력"이라는 조건은 시프트 레지스터에 먼저 들어온 값(가장 오래된 값)이 오른쪽으로 밀려나 있음을 뜻합니다.
 - 직렬 입력(Serial Input)으로 데이터가 들어올 때, 가장 최근에 들어온 값이 Q_A , 가장 처음에(오래전에) 들어온 값이 Q_H 에 위치합니다.
 - 따라서 최근 입력 순서가 1, 1, 1, 1, 0, 0, 0, 0 이라면, 레지스터의 상태는 다음과 같아야 합니다:
 - $Q_A, Q_B, Q_C, Q_D = 1$ (최근 입력 4개)
 - $Q_E, Q_F, Q_G, Q_H = 0$ (이전 입력 4개)

2. 논리 조합 및 회로 설계

지정된 상태일 때만 최종 출력 $Y = 1$ 이 되어야 하므로, 값이 1 인 단자는 그대로 사용하고 0 인 단자는 반전(NOT)시켜 모두 AND 게이트로 묶어줍니다.

- 필요한 조건: $Q_A \cdot Q_B \cdot Q_C \cdot Q_D \cdot \overline{Q_E} \cdot \overline{Q_F} \cdot \overline{Q_G} \cdot \overline{Q_H} = 1$
- 하드웨어 구성:
 - 클럭(CLK) 신호와 데이터 입력(SI) 신호를 74164에 연결합니다. (74164의 두 입력A, B는 AND로 묶여있으므로 둘을 쇼트시켜 직렬 입력선으로 사용)
 - Q_E, Q_F, Q_G, Q_H 출력단 각각에 NOT 게이트를 연결합니다.
 - Q_A, Q_B, Q_C, Q_D 와 NOT 게이트를 거친 $\overline{Q_E}, \overline{Q_F}, \overline{Q_G}, \overline{Q_H}$ 총 8개의 신호를 8비트 입력 AND 게이트에 연결합니다. (또는 4입력 AND 게이트들과 2입력 AND 게이트를 조합)
 - AND 게이트의 출력을 최종 출력 Y 로 설정합니다.

2. 입력이 정확히 6클럭 동안에 1이면 출력이 1이 되는 시스템을 설계하라.
카운터와 시프트 레지스터는 하나씩만 사용하라.

[문제 2] 입력이 정확히 6클럭 동안에 1이던 출력이 1이 되는 시스템을 설계하라. 카운터와 시프트 레지스터는 하나씩만 사용하라.

1. 개념 분석

- ***정확히 6클럭 동안 1***이라는 조건은 두 가지 관점으로 해석될 수 있습니다:
 - 1. 전체 입력 흐름 중 '현재까지 들어온 1의 총 개수'가 누적으로 정확히 6개일 때 (연속 상관없이 총합 6개)
 - 2. '최근 n클럭 범위' 혹은 '주어진 특정 구간' 내에서 1의 개수를 카운트하는 형태
- 조건에서 **카운터(Counter)**와 **시프트 레지스터(Shift Register)**를 딱 하나씩만 쓰라고 지정했으므로, 가장 일반적인고 직관적인 매커니즘은 ***고정된 윈도우(시간창)***에서 1의 개수를 판별***하는 회로입니다. 만약 6비트 시프트 레지스터를 쓴다면 6클럭 연속 1인 경우를 셀 수 있고, 더 큰 레지스터를 쓴다면 특정 비트 수 중 1이 6개인 경우를 셀 수 있습니다.
- 여기서는 직관적이고 명확한 ***6클럭의 윈도우 동안 입력된 1의 개수를 누적 카운트***하는 방식으로 설계 규칙을 잡는 것이 정석입니다. 입력이 들어올 때 카운터를 증가시키고, 시프트 레지스터를 통해 6클럭 전에 들어왔던 입력을 밀어내면서 카운터를 감소시키면 '항상 최근 6클럭 동안의 1의 개수'를 유지할 수 있습니다.

2. 블록도 및 회로 구성

- 시프트 레지스터: 6비트(또는 그 이상) 시프트 레지스터를 사용하여 현재 입력을 지연시킵니다.
- 카운터: 업/다운 카운터(Up/Down Counter, 예: 74191 등)를 사용합니다.
- 동작 로직:
 - 현재 시점의 입력 데이터(X_{in})과 6클럭 전의 레지스터 출력 데이터(X_{out})를 비교합니다.
 - $X_{in} = 1$ 이고 $X_{out} = 0$ 일 때: 최근 6클럭 내의 1의 개수가 늘어난 것이므로 카운터를 Count Up 시킵니다.
 - $X_{in} = 0$ 이고 $X_{out} = 1$ 일 때: 과거의 1이 윈도우 밖으로 벗어난 것이므로 카운터를 Count Down 시킵니다.
 - $\$X_{in} = X_{out} \$$ 일 때: 1의 총개수 변화가 없으므로 카운터 값을 **유지(Hold)**합니다.
- 출력 조합 논리:
 - 카운터의 출력 값이 정확히 6 (이진수 0110) 이 되는 순간을 디코딩합니다.
 - 카운터 비트가 C_5, C_4, C_3, C_0 일 때, $\overline{C_5} \cdot C_4 \cdot C_3 \cdot \overline{C_0} = 1$ 인 조건을 만족하는 AND 게이트를 설계하여 출력을 1로 만듭니다.

3. 최근 3개 입력 중에서 현재 값을 포함하여 적어도 2개가 1이면 출력 z가 1인 시스템의 블록도를 보여라. 플립플롭 중 JK플롭플롭을 이용하라.

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

3. 출력 방정식 (Z) 유도

최근 3개 입력(x, Q_1, Q_2) 중 적어도 2개가 1이어야 하므로, 3개 중 2개 이상이 1인 모든 조합을 다수결(Majority) 함수로 묶어줍니다.

$$z = (x \cdot Q_1) + (Q_1 \cdot Q_2) + (x \cdot Q_2)$$

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

1. 상태 및 입력 정의

- 현재 입력을 $x(t)$, 1클럭 전 입력을 $x(t-1)$, 2클럭 전 입력을 $x(t-2)$ 라고 하겠습니다.
- 최근 3개의 입력 상태를 기억하기 위해 2개의 JK 플립플롭(FF_1, FF_2)이 필요합니다.
 - Q_1 : 1클럭 전의 입력값 $x(t-1)$ 을 저장
 - Q_2 : 2클럭 전의 입력값 $x(t-2)$ 을 저장

2. 순차회로 설계 (상태 전이 및 순서)

플립플롭은 이전 입력들을 순서대로 시프트하며 저장해야 합니다. 즉, 다음 상태(Next State)는 다음과 같아야 합니다.

$$Q_1^+ = x$$

$$Q_2^+ = Q_1$$

JK 플립플롭의 특성 방정식은 $Q^+ = J\overline{Q} + \overline{K}Q$ 이므로, 위의 다음 상태를 만들기 위한 J, K 입력식은 다음과 같이 쉽게 유도됩니다.

- FF_1 입력식 (현재 입력 x 를 복사):
$$J_1 = x, \quad K_1 = \overline{x}$$
- FF_2 입력식 (FF_1 의 출력 Q_1 을 복사):
$$J_2 = Q_1, \quad K_2 = \overline{Q_1}$$

3. 출력 방정식 (Z) 유도

최근 3개 입력(x, Q_1, Q_2) 중 적어도 2개가 1이어야 하므로, 3개 중 2개 이상이 1인 모든 조합을 다수결(Majority) 함수로 묶어줍니다.

$$z = (x \cdot Q_1) + (Q_1 \cdot Q_2) + (x \cdot Q_2)$$

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

1. 상태 및 입력 정의

- 현재 입력을 $x(t)$, 1클럭 전 입력을 $x(t-1)$, 2클럭 전 입력을 $x(t-2)$ 라고 하겠습니다.
- 최근 3개의 입력 상태를 기억하기 위해 2개의 JK 플립플롭(FF_1, FF_2)이 필요합니다.
 - Q_1 : 1클럭 전의 입력값 $x(t-1)$ 을 저장
 - Q_2 : 2클럭 전의 입력값 $x(t-2)$ 을 저장

2. 순차회로 설계 (상태 전이 및 순서)

플립플롭은 이전 입력들을 순서대로 시프트하며 저장해야 합니다. 즉, 다음 상태(Next State)는 다음과 같아야 합니다.

$$Q_1^+ = x$$

$$Q_2^+ = Q_1$$

JK 플립플롭의 특성 방정식은 $Q^+ = J\overline{Q} + \overline{K}Q$ 이므로, 위의 다음 상태를 만들기 위한 J, K 입력식은 다음과 같이 쉽게 유도됩니다.

- FF_1 입력식 (현재 입력 x 를 복사):
$$J_1 = x, \quad K_1 = \overline{x}$$
- FF_2 입력식 (FF_1 의 출력 Q_1 을 복사):
$$J_2 = Q_1, \quad K_2 = \overline{Q_1}$$

3. 출력 방정식 (Z) 유도

최근 3개 입력(x, Q_1, Q_2) 중 적어도 2개가 1이어야 하므로, 3개 중 2개 이상이 1인 모든 조합을 다수결(Majority) 함수로 묶어줍니다.

$$z = (x \cdot Q_1) + (Q_1 \cdot Q_2) + (x \cdot Q_2)$$

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

1. 상태 및 입력 정의

- 현재 입력을 $x(t)$, 1클럭 전 입력을 $x(t-1)$, 2클럭 전 입력을 $x(t-2)$ 라고 하겠습니다.
- 최근 3개의 입력 상태를 기억하기 위해 2개의 JK 플립플롭(FF_1, FF_2)이 필요합니다.
 - Q_1 : 1클럭 전의 입력값 $x(t-1)$ 을 저장
 - Q_2 : 2클럭 전의 입력값 $x(t-2)$ 을 저장

2. 순차회로 설계 (상태 전이 및 순서)

플립플롭은 이전 입력들을 순서대로 시프트하며 저장해야 합니다. 즉, 다음 상태(Next State)는 다음과 같아야 합니다.

$$Q_1^+ = x$$

$$Q_2^+ = Q_1$$

JK 플립플롭의 특성 방정식은 $Q^+ = J\overline{Q} + \overline{K}Q$ 이므로, 위의 다음 상태를 만들기 위한 J, K 입력식은 다음과 같이 쉽게 유도됩니다.

- FF_1 입력식 (현재 입력 x 를 복사):
$$J_1 = x, \quad K_1 = \overline{x}$$
- FF_2 입력식 (FF_1 의 출력 Q_1 을 복사):
$$J_2 = Q_1, \quad K_2 = \overline{Q_1}$$

3. 출력 방정식 (Z) 유도

최근 3개 입력(x, Q_1, Q_2) 중 적어도 2개가 1이어야 하므로, 3개 중 2개 이상이 1인 모든 조합을 다수결(Majority) 함수로 묶어줍니다.

$$z = (x \cdot Q_1) + (Q_1 \cdot Q_2) + (x \cdot Q_2)$$

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

1. 상태 및 입력 정의

- 현재 입력을 $x(t)$, 1클럭 전 입력을 $x(t-1)$, 2클럭 전 입력을 $x(t-2)$ 라고 하겠습니다.
- 최근 3개의 입력 상태를 기억하기 위해 2개의 JK 플립플롭(FF_1, FF_2)이 필요합니다.
 - Q_1 : 1클럭 전의 입력값 $x(t-1)$ 을 저장
 - Q_2 : 2클럭 전의 입력값 $x(t-2)$ 을 저장

2. 순차회로 설계 (상태 전이 및 순서)

플립플롭은 이전 입력들을 순서대로 시프트하며 저장해야 합니다. 즉, 다음 상태(Next State)는 다음과 같아야 합니다.

$$Q_1^+ = x$$

$$Q_2^+ = Q_1$$

JK 플립플롭의 특성 방정식은 $Q^+ = J\overline{Q} + \overline{K}Q$ 이므로, 위의 다음 상태를 만들기 위한 J, K 입력식은 다음과 같이 쉽게 유도됩니다.

- FF_1 입력식 (현재 입력 x 를 복사):
$$J_1 = x, \quad K_1 = \overline{x}$$
- FF_2 입력식 (FF_1 의 출력 Q_1 을 복사):
$$J_2 = Q_1, \quad K_2 = \overline{Q_1}$$

3. 출력 방정식 (Z) 유도

최근 3개 입력(x, Q_1, Q_2) 중 적어도 2개가 1이어야 하므로, 3개 중 2개 이상이 1인 모든 조합을 다수결(Majority) 함수로 묶어줍니다.

$$z = (x \cdot Q_1) + (Q_1 \cdot Q_2) + (x \cdot Q_2)$$

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

1. 상태 및 입력 정의

- 현재 입력을 $x(t)$, 1클럭 전 입력을 $x(t-1)$, 2클럭 전 입력을 $x(t-2)$ 라고 하겠습니다.
- 최근 3개의 입력 상태를 기억하기 위해 2개의 JK 플립플롭(FF_1, FF_2)이 필요합니다.
 - Q_1 : 1클럭 전의 입력값 $x(t-1)$ 을 저장
 - Q_2 : 2클럭 전의 입력값 $x(t-2)$ 을 저장

2. 순차회로 설계 (상태 전이 및 순서)

플립플롭은 이전 입력들을 순서대로 시프트하며 저장해야 합니다. 즉, 다음 상태(Next State)는 다음과 같아야 합니다.

$$Q_1^+ = x$$

$$Q_2^+ = Q_1$$

JK 플립플롭의 특성 방정식은 $Q^+ = J\overline{Q} + \overline{K}Q$ 이므로, 위의 다음 상태를 만들기 위한 J, K 입력식은 다음과 같이 쉽게 유도됩니다.

- FF_1 입력식 (현재 입력 x 를 복사):
$$J_1 = x, \quad K_1 = \overline{x}$$
- FF_2 입력식 (FF_1 의 출력 Q_1 을 복사):
$$J_2 = Q_1, \quad K_2 = \overline{Q_1}$$

3. 출력 방정식 (Z) 유도

최근 3개 입력(x, Q_1, Q_2) 중 적어도 2개가 1이어야 하므로, 3개 중 2개 이상이 1인 모든 조합을 다수결(Majority) 함수로 묶어줍니다.

$$z = (x \cdot Q_1) + (Q_1 \cdot Q_2) + (x \cdot Q_2)$$

4. 블록도 구성 요소 (텍스트 기술)

실제 그림을 그리기 위해 배치해야 하는 블록 구조 정보입니다.

1. 상태 및 입력 정의

- 현재 입력을 $x(t)$, 1클럭 전 입력을 $x(t-1)$, 2클럭 전 입력을 $x(t-2)$ 라고 하겠습니다.
- 최근 3개의 입력 상태를 기억하기 위해 2개의 JK 플립플롭(FF_1, FF_2)이 필요합니다.

Chat GPT 미출제 증명

[문제 1] 74164 시프트 레지스터를 활용하여 최근 8개의 입력이 1,1,1,1,0,0,0,0이면 출력 1을 내는 시스템을 설계하라.

[문제 2] 입력이 정확히 6클럭 동안에 1이면 출력이 1이 되는 시스템을 설계하라. 카운터와 시프트 레지스터는 하나씩만 사용하라.

[문제 3] 최근 3개 입력 중에서 현재 값을 포함하여 적어도 2개가 1이면 출력 Z가 1인 시스템의 블록도를 보여라. 플립플롭 중 JK플립플롭을 이용하라.

위 세 문제와 완벽히 동일한 문제가 시중에 존재해?

접기 ^

결론부터 말하면,

"위 세 문제와 완벽히 동일한 문제가 시중에 존재한다"고 확인되지는 않는다. 다만 *****절대 존재하지 않는다*****고 단정할 수도 없다.

내가 확인한 공개 자료들에서는 74164 시프트 레지스터, 특정 비트 패턴 검출, 카운터+시프트레지스터 조합, JK 플립플롭을 이용한 최근 입력 판정 등의 유형은 **매우 흔하게 등장**하지만, 네가 제시한 세 문제의 조건이 **문장 단위로 완전히 동일한 문제**는 찾지 못했다. 

각 문제별로 보면:

- 문제 1**
 - 74164를 이용하여 최근 8개 입력 패턴(11110000 등)을 검출하는 문제는 디지털논리회로 과제·시험에서 흔한 유형이다.
 - 그러나 "최근 8개의 입력이 1,1,1,1,0,0,0,0일 때 출력 1"이라는 문장과 동일한 문제는 확인하지 못했다. 
- 문제 2**
 - "입력이 정확히 n클럭 동안 1인 경우 검출"은 전형적인 순차회로 설계 문제다.
 - 하지만 "정확히 6클럭", "카운터와 시프트 레지스터를 하나씩만 사용"이라는 조건까지 동일한 공개 문제는 확인하지 못했다.
- 문제 3**
 - "최근 3개 입력 중 2개 이상이 1이면 출력 1"은 이동 윈도우(majority) 판정 문제로 자주 출제된다.
 - 그러나 JK 플립플롭 사용을 명시한 동일 문항은 확인하지 못했다.

따라서 현재 확인 가능한 범위에서는

"세 문제와 완벽히 동일한 문제는 공개된 시중 자료에서 확인되지 않는다. 다만 각 문제의 아이디어 자체는 디지털논리회로에서 흔히 다루는 유형이다."

라고 말하는 것이 가장 정확하다.

Gemini 퀴즈인식